Поисковые модели

Часть 1: Семантический поиск

План лекции

- Информационный поиск и типы задач
- Архитектуры ранжирующих моделей семантического поиска
- Представления текста и подготовка данных
- Функции потерь
- Негативное семплирование и организация обучения
- Метрики и анализ ошибок
- Итоги и задание первой лабораторной

Информационный поиск

1. Получение скоров

$$s_i = f_\theta(q, d_i)$$

Где q - запрос, C - корпус, d_i - документ;
θ - параметры функции f, s_i - скор.

2. Ранжирование

$$\pi = \text{argsort}^{\downarrow}(s_1, \ldots, s_N)$$

Где N - число документов;
π(r) - индекс документа на позиции r.

3. Усечение выдачи

$$A(q) = (d_{\pi(1)}, \ldots, d_{\pi(K)})$$

Где A(q) - выдача, K - число результатов;
k - порог по рангу, т - минимальный скор.

По рангу: первые k документов (K = k).

По скору: документы со скором ≥ т.

Как восстановить удалённый файл?

Ранг	Кандидат	Скор	Метка y
1	Восстановление из корзины	0.86	**1**
2	Безвозвратное удаление файлов	0.79	**0**
		т = 0.70 (по скору)	
3	Восстановление резервной копии	0.68	**1**
		k = 3 (по рангу)	
4	Переименование файлов	0.42	**0**
5	Восстановление через историю версий	0.27	**1**

Задачи информационного поиска

Задача	Входные данные	Цель обучения
Целевые задачи (fine-tuning)		
IR	Запрос - документ	Релевантность документа
QA	Вопрос - ответ	Соответствие ответа вопросу
STS	Текст - текст	Семантическое сходство
Вспомогательные задачи (proxy)		
Title-Body	Заголовок - текст	Соответствие заголовка содержанию
Summary-Full Text	Резюме - полный текст	Соответствие краткого и полного содержания
Post-Reply	Сообщение - ответ	Содержательная связь реплик
NLI	Посылка - гипотеза	Следование, противоречие, нейтральность
Предобучение (self-supervised)		
ICT	Предложение - контекст	Поиск исходного контекста
NSP	Фрагмент - фрагмент	Определение соседства фрагментов
MLM	Текст с пропусками	Восстановление токенов
SimCSE (unsup.)	Текст - тот же текст	Близость представлений при разном dropout
Contriever	Фрагмент - фрагмент	Близость фрагментов одного документа

Validation setups

Queries	Candidates
query 1	candidate 1
query 2	candidate 2
…	…
query n	candidate m

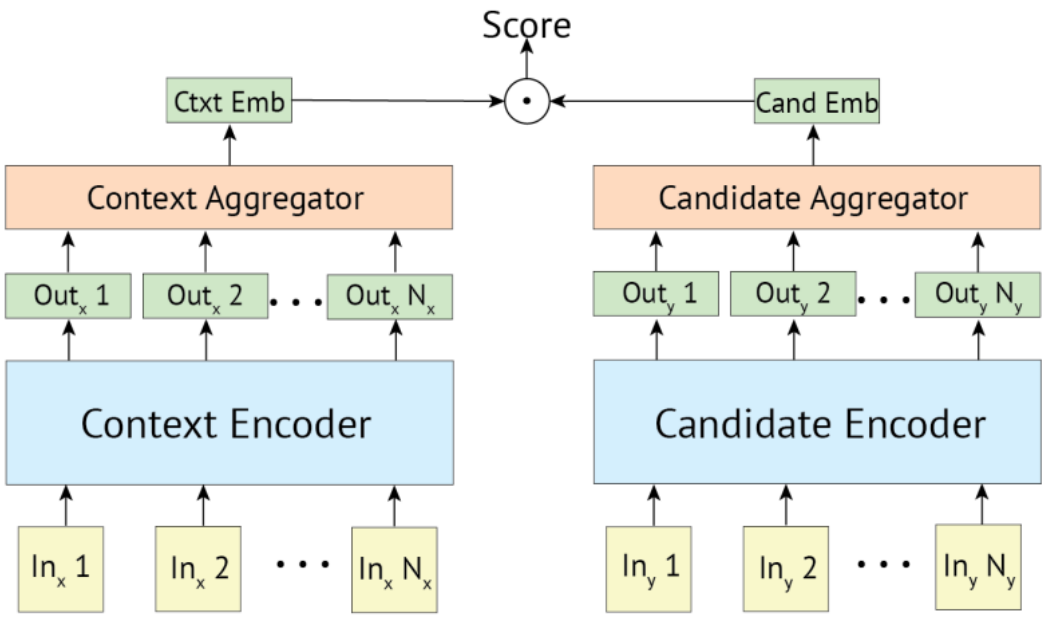
n>m, m>n, n=m, n!=m

query i – candidate i
query i – [candidate i,…, candidate j]
[query i, …, query j] – candidate i
[query i, …, query j] – [candidate i,…, candidate j]

Архитектуры ранжирующих моделей

Семантические поисковые модели

Bi-Encoder



(a) Bi-encoder

Векторы документов вычисляются заранее. Запрос кодируется при поиске.

[Karpukhin et al. DPR, 2020 - arxiv.org/abs/2004.04906](arxiv.org/abs/2004.04906)

Независимое кодирование

$z_q = E_Q(q) \in \mathbb{R}^h$

$z_d = E_D(d) \in \mathbb{R}^h$

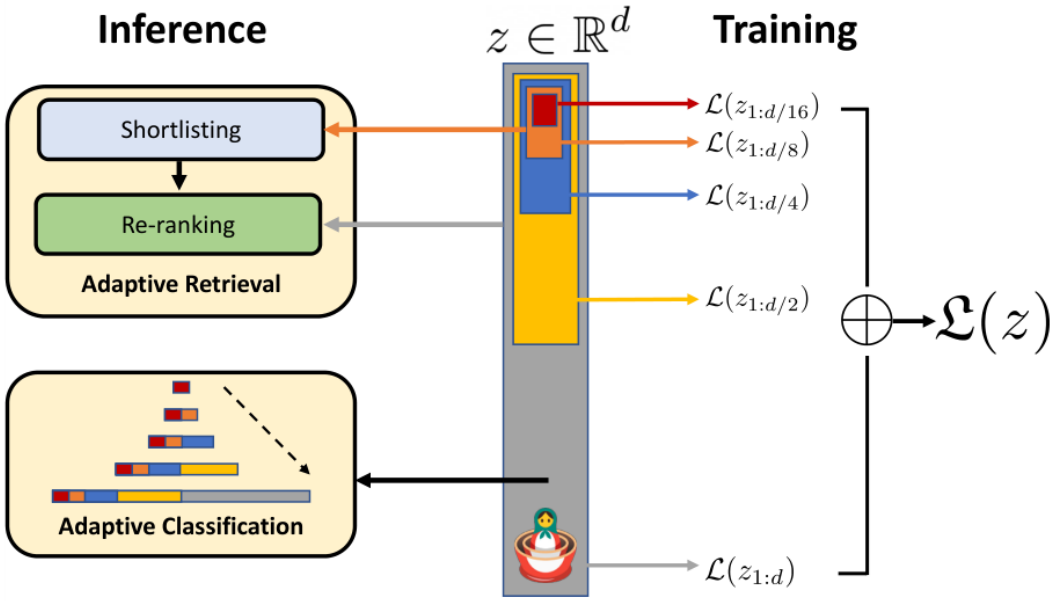
Скор релевантности

$s(q,d) = z_q^T z_d$

Где q - запрос, d - документ,
E_Q и E_D - энкодеры,
z_q и z_d - векторы размерности h.

DPR: два BERT-энкодера
с отдельными весами.

Bi-Encoder: Matryoshka



Префикс и скор

$z = f(x) \in \mathbb{R}^D$

$z^{(m)} = z_{1:m} / \|z_{1:m}\|_2$

$s_m(q,d) = z_q^{(m)T} z_d^{(m)}$

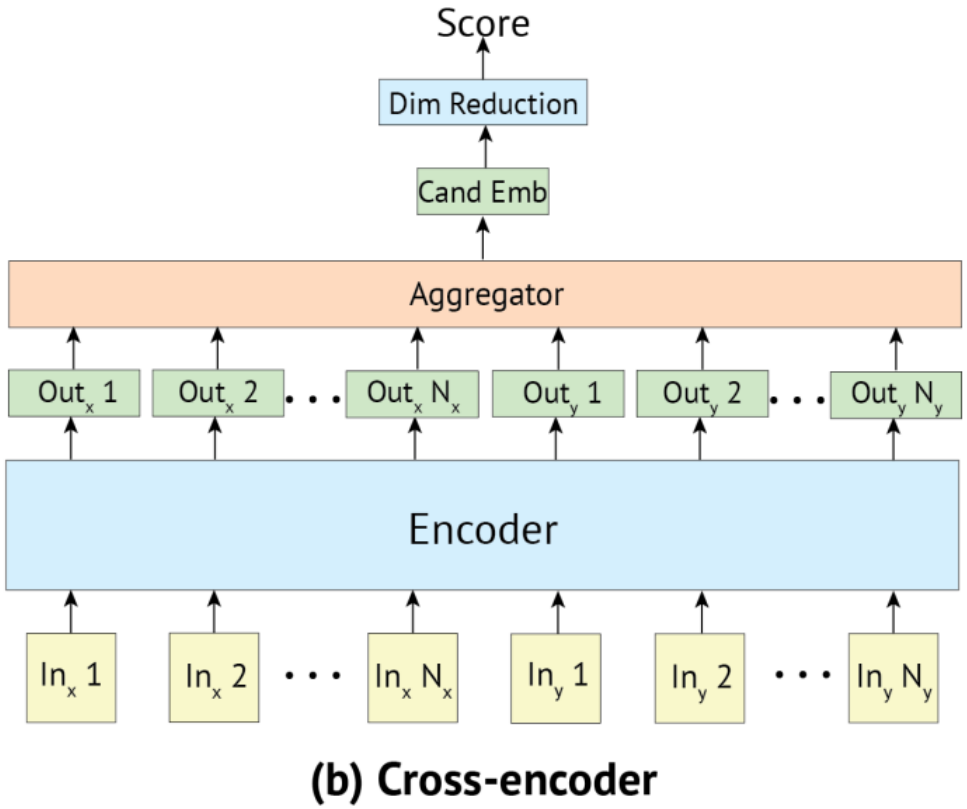
Где $m \in M$ - длина префикса,
$M \subseteq \{1,\ldots,D\}$ - набор размерностей.
Нормировка выполняется после усечения.

$L_{MRL} = \sum_{m \in M} \lambda_m L_{retrieval}(s_m)$

Где $L_{retrieval}$ - loss поиска на размерности m, λ_m - вес. Один энкодер обучается для всех m.

[Kusupati et al. Matryoshka Representation Learning, 2022 - arxiv.org/abs/2205.13147](arxiv.org/abs/2205.13147)

Cross-Encoder



(b) Cross-encoder

Токены двух текстов взаимодействуют.

Совместная оценка пары

$$s_\vartheta(q,d) = h_\vartheta([q; d])$$

Где [q; d] - объединение текстов,
h_θ - модель скоринга с параметрами θ.

Реализация с BERT

$$x = [CLS]\ q\ [SEP]\ d\ [SEP]$$

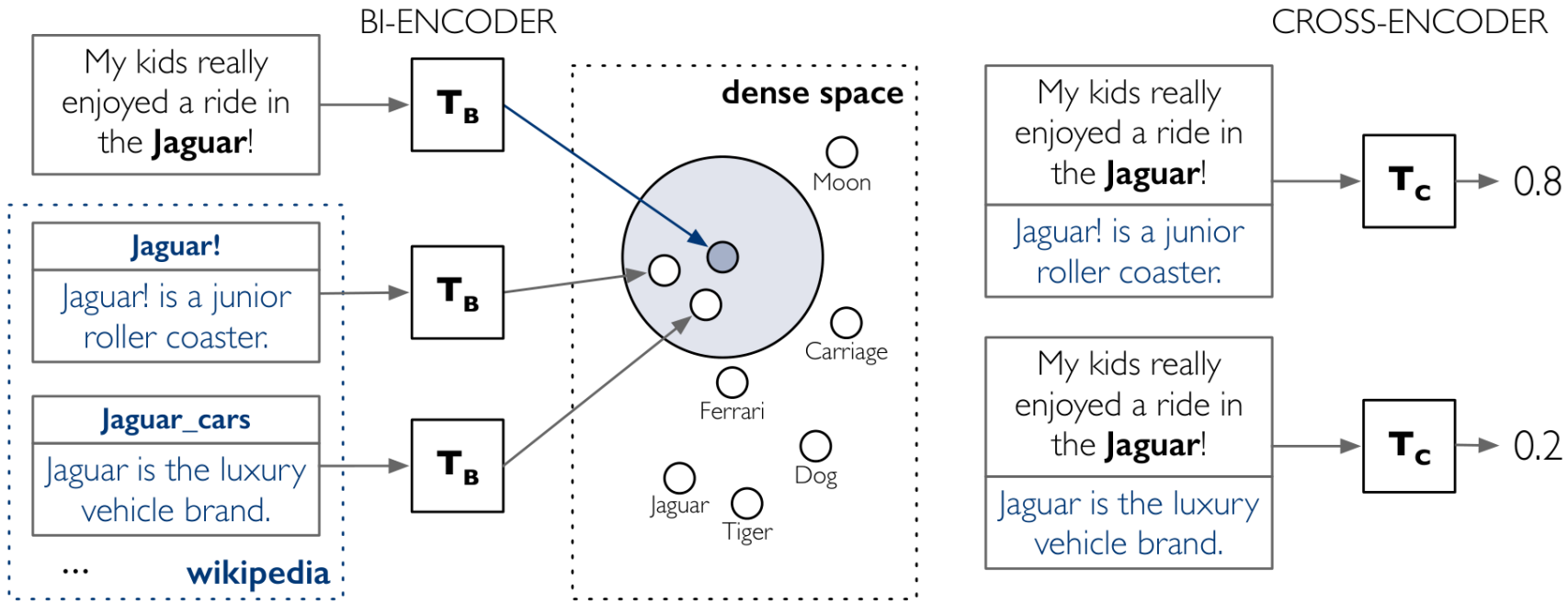
$$h = BERT(x)_{[CLS]}$$

$$s_\vartheta(q,d) = \sigma(w^T h + b)$$

Где h - CLS-вектор пары,
w и b - параметры классификатора,
σ - сигмоида.

Nogueira, Cho. Passage Re-ranking with BERT, 2019 - arxiv.org/abs/1901.04085

Поиск и реранжирование



1. Отбор биэнкодером

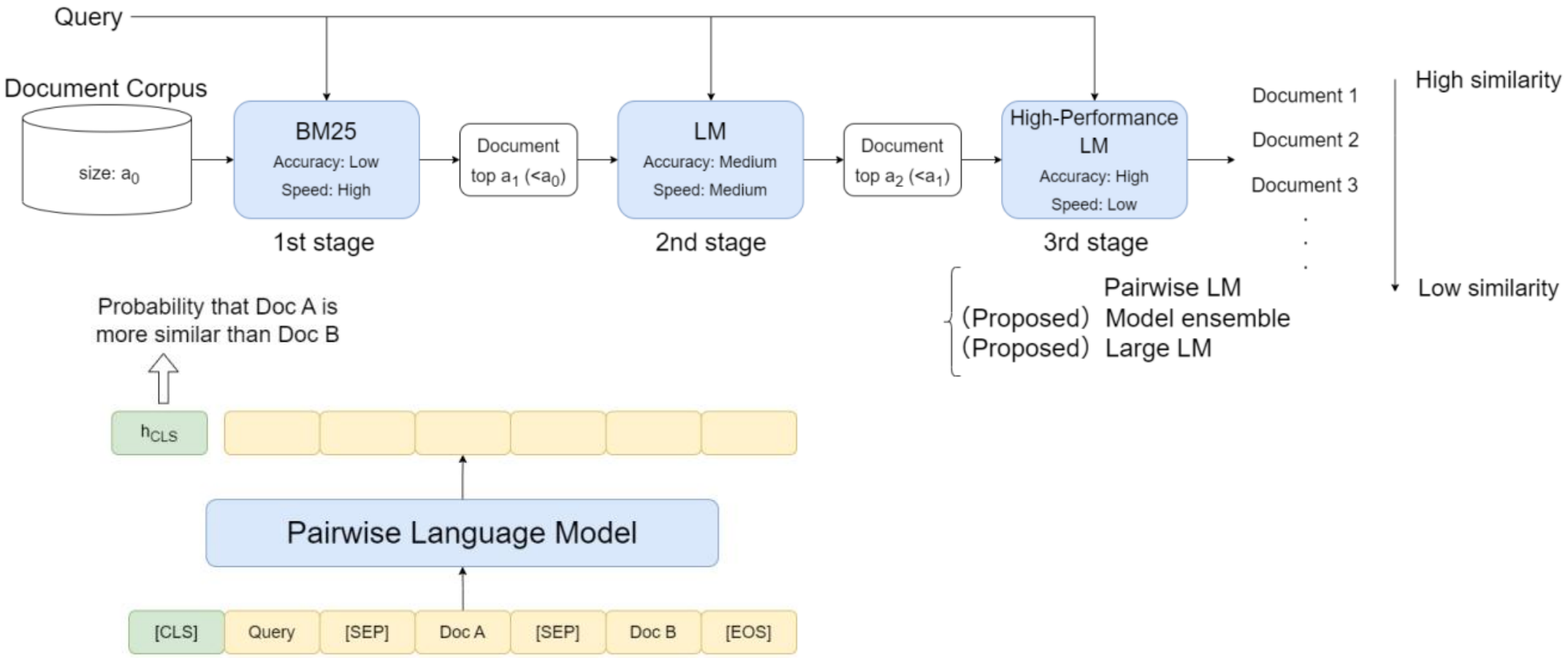
Из корпуса выбираются k кандидатов.
Больше k - выше охват и стоимость.

2. Реранжирование

Кроссэнкодер уточняет порядок.
Пропущенный кандидат не вернётся.

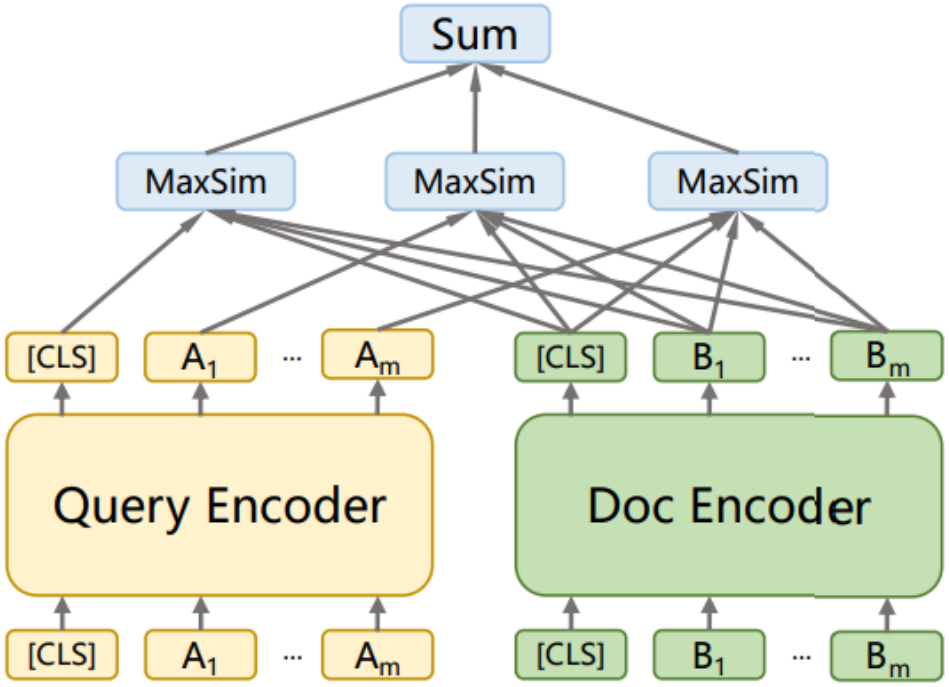
Wu et al. BLINK, 2020. Рис. 1: пример entity linking - arxiv.org/abs/1911.03814

Ladder reranking



[Многоэтапное ранжирование: Nogueira et al., 2019 - arxiv.org/abs/1910.14424]

ColBERT



(c) Late Interaction (e.g., ColBERT)

Позднее взаимодействие: MaxSim

$s(q,d) = \sum_{i=1}^{m} \max_{1 \le j \le n} q_i^T d_j$

Где q_i и d_j - нормированные векторы токенов, m и n - число векторов запроса и документа.

Для каждого токена запроса выбирается самое близкое соответствие в документе. Эти сходства суммируются.

Документ хранит много векторов. Их можно вычислить заранее.

Khattab, Zaharia. ColBERT, SIGIR 2020 - arxiv.org/abs/2004.12832

ColBERTv2

Сжатие вектора

$t = \text{argmin}_j \|v - c_j\|_2^2$

$r = v - c_t, \quad \tilde{r} = Q_b(r)$

$\tilde{v} = c_t + \tilde{r}$

Где c_j - центроид, Q_b - квантование остатка с b битами на координату.

$\text{bytes}(v) = 4 + 128b / 8$

b = 1: 20 байт. b = 2: 36 байт.
Исходный вектор FP16: 256 байт.

Дистилляция скоров

$p_T = \text{softmax}(s_T)$,

$p_S = \text{softmax}(s_S)$

$L_{KD} = \sum_{i=1}^{k} p_{T,i} \ln(p_{T,i} / p_{S,i})$

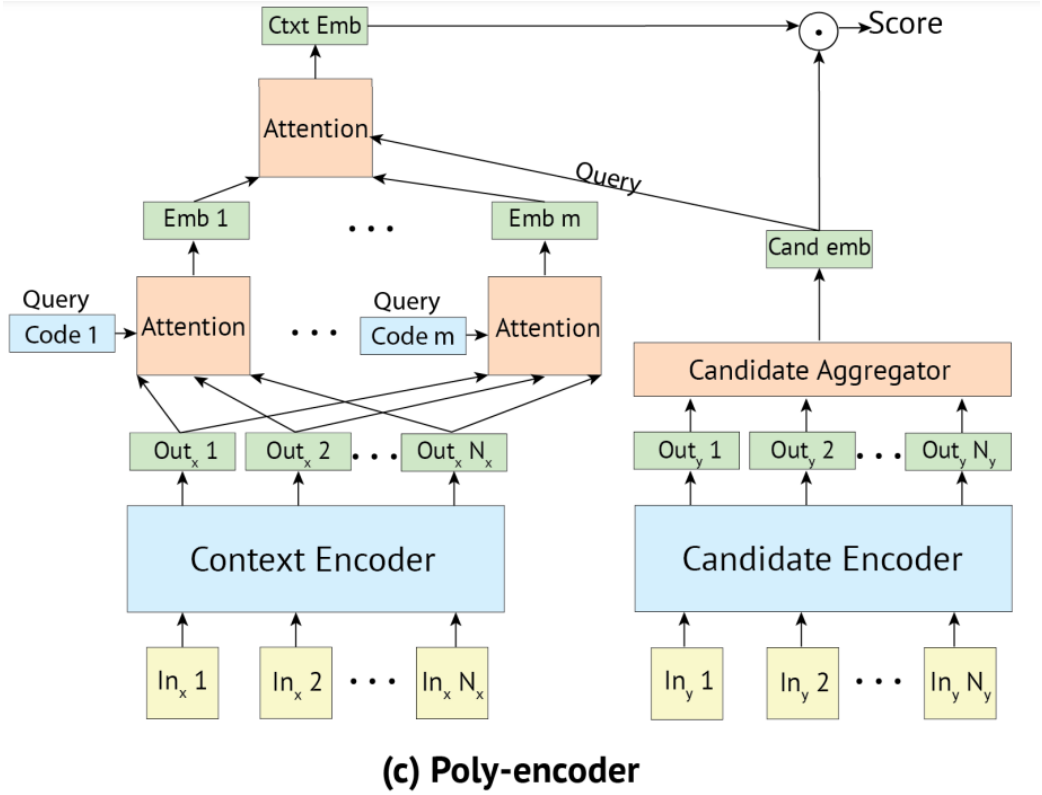
$L = L_{KD} + L_{\text{in-batch}}$

Где T - Cross-Encoder, S - ColBERTv2,
s_T и s_S - скоры k кандидатов,
p_T и p_S - распределения по ним.
$L_{\text{in-batch}}$ - loss с негативами батча.

MaxSim сохраняется. Размер индекса в экспериментах статьи уменьшается в 6-10 раз.

Santhanam et al. ColBERTv2, NAACL 2022 - arxiv.org/abs/2112.01488

Poly-Encoder



(c) Poly-encoder

1. Представления запроса

$$\beta_r = \text{softmax}(H_q a_r)$$

$$c_r = H_q^T \beta_r, \quad r = 1,\ldots,m$$

Где H_q - матрица векторов токенов,
a_r - обучаемый код, β_r - веса attention,
c_r - одно из m представлений.

2. Взаимодействие с документом

$$\alpha = \text{softmax}(C v_d)$$

$$s(q,d) = v_d^T \sum_{r=1}^{m} \alpha_r c_r$$

Где C содержит строки c_r^T,
v_d - вектор документа, α_r - вес c_r.

Humeau et al. Poly-encoders, ICLR 2020 - arxiv.org/abs/1905.01969

Poly-Encoder: число кодов

ConvAI2: качество и время на GPU

Модель	R@1/20, %	1k, мс	100k, мс
Bi-Encoder	81,7	19	22
Poly, m=16	83,2	18	38
Poly, m=64	83,7	23	46
Poly, m=360	83,7	57	88
Cross-Encoder	84,8	2 600	266 000*

Качество: BERT, test, 20 кандидатов.
Время: 1k/100k кандидатов, Quadro GP100.
* Оценка авторов. Таблицы 4 и 5.

Humeau et al. Poly-encoders, ICLR 2020 - arxiv.org/abs/1905.01969

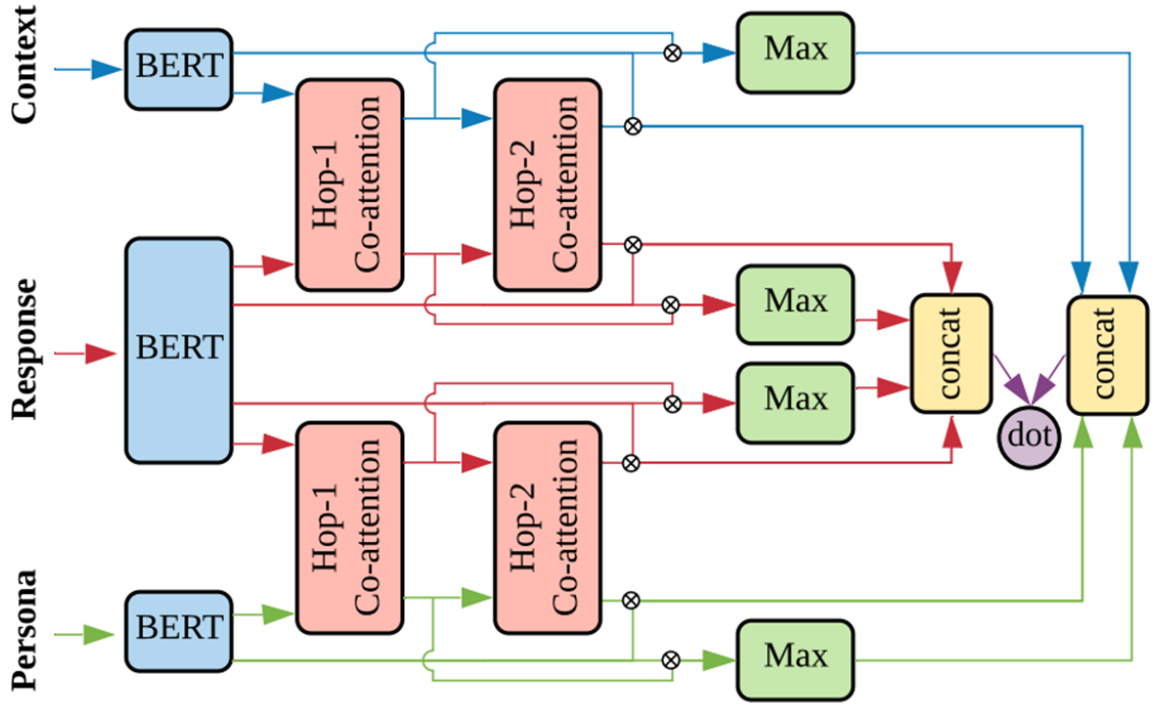
Параметр m

Больше кодов - больше представлений контекста и больше операций attention.

В этом эксперименте рост с 64 до 360 кодов не улучшил R@1/20, но увеличил время.

Число кодов выбирают по качеству на validation и допустимой задержке.

CoBERT



Независимое кодирование

X = E(c), P = E(p),

Y = E(r)

Скор после co-attention

u = $[x_1; x_2; p_1; p_2]$

v = $[y_1^c; y_2^c; y_1^p; y_2^p]$

s(c,p,r) = $u^T v$

Где [;] - конкатенация,
1 и 2 - шаги co-attention.

Где c - контекст диалога, p - персона, r - ответ. E - BERT, X, P, Y - матрицы векторов токенов.

Zhong et al. CoBERT, EMNLP 2020 - aclanthology.org/2020.emnlp-main.531/

CoBERT: два шага co-attention

Hop-1: сопоставление токенов

$A = XY^T$

$B = \text{softmax}(A)$,

$C = \text{softmax}(A^T)$

$x_1 = \text{maxpool}(BY)$,

$y_1^c = \text{maxpool}(CX)$

Softmax нормирует строки.
Max pooling выполняется по токенам.

Hop-2: attention по attention

$a_X = \text{mean}(B)C$,

$a_Y = \text{mean}(C)B$

$x_2 = a_X X$,

$y_2^c = a_Y Y$

Mean pooling усредняет строки.
Веса второго шага учитывают attention в обоих направлениях.

Где X и Y - матрицы токенов контекста и ответа. Для персоны P выполняются те же операции.
Hop-1 даёт локальные соответствия, Hop-2 агрегирует информацию о взаимном внимании.

Zhong et al. CoBERT, EMNLP 2020 - aclanthology.org/2020.emnlp-main.531/

Краткие выводы

Модель	**Плюсы**	**Минусы**
Bi-Encoder	Быстрый поиск по векторному индексу Документы кодируются заранее	Один вектор сжимает весь текст Нет взаимодействия токенов пары
Cross-Encoder	Совместный attention токенов Моделирует связи внутри пары	Энкодер запускается на каждой паре Дорогой поиск по всему корпусу
ColBERT	Сопоставление токенов через MaxSim Векторы документов готовы заранее	Больше памяти на индекс Больше операций, чем у Bi-Encoder
Poly-Encoder	Несколько представлений запроса Векторы документов готовы заранее	Attention для каждого кандидата Документ сжат в один вектор
CoBERT	Два шага взаимодействия токенов Учитывает контекст и персону	Скор требует обработки пары Дороже сравнения готовых векторов

Обучение ранжирующей модели

На примере Bi-Encoder

Получение вектора текста

Mean pooling с маской padding

$$z = (\sum_{i=1}^{L} a_i h_i) / (\sum_{i=1}^{L} a_i)$$

Где h_i - выход энкодера для токена i, a_i - маска непустых позиций, L - длина входа.

Путь текста к вектору

Токенизация задаёт входные токены, attention mask и усечение длинных текстов.

Энкодер вычисляет контекстные векторы. Pooling объединяет их в один вектор.

Выбор pooling

Mean, CLS и последний токен дают разные представления. Выбор должен соответствовать обучению модели.

После pooling можно применить L2-нормализацию для cosine similarity.

Функции подобия и расстояния

Косинусное подобие

$$s_{cos}(u, v) = \frac{u^T v}{\|u\|_2 \|v\|_2}$$

Сравнивает направления векторов.
Не зависит от их длины.

Скалярное произведение

$$s_{dot}(u, v) = u^T v$$

Учитывает направление
и длину векторов.

Евклидово расстояние (L2)

$$d_2(u, v) = \sqrt{\sum_{i=1}^{h} (u_i - v_i)^2}$$

Длина прямого отрезка
между точками.

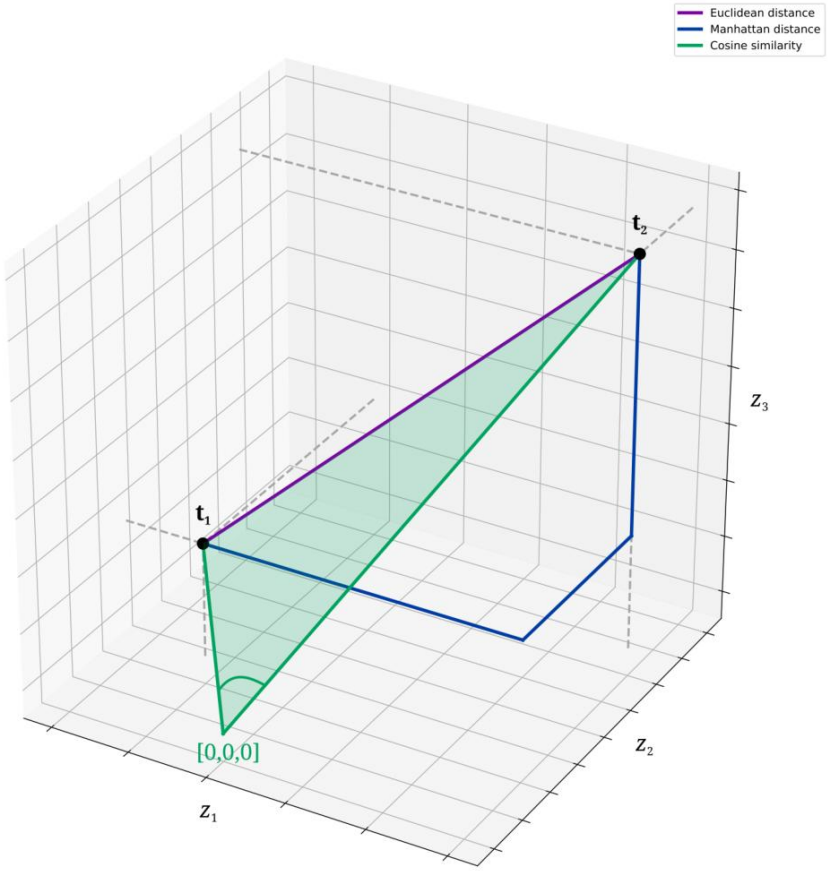
Манхэттенское расстояние (L1)

$$d_1(u, v) = \sum_{i=1}^{h} |u_i - v_i|$$

Сумма абсолютных разностей
по координатам.

Где u и v - векторы размерности h. Подобие максимизируют, расстояние минимизируют.
Расстояние можно превратить в скор, поменяв знак: s(u, v) = -d(u, v).

Геометрия подобия и расстояний



L2: прямая между точками
Кратчайший путь в пространстве.

L1: путь вдоль координат
Сумма длин отдельных шагов.

Cosine: угол между векторами
Длины векторов не учитываются.

Для единичных векторов:

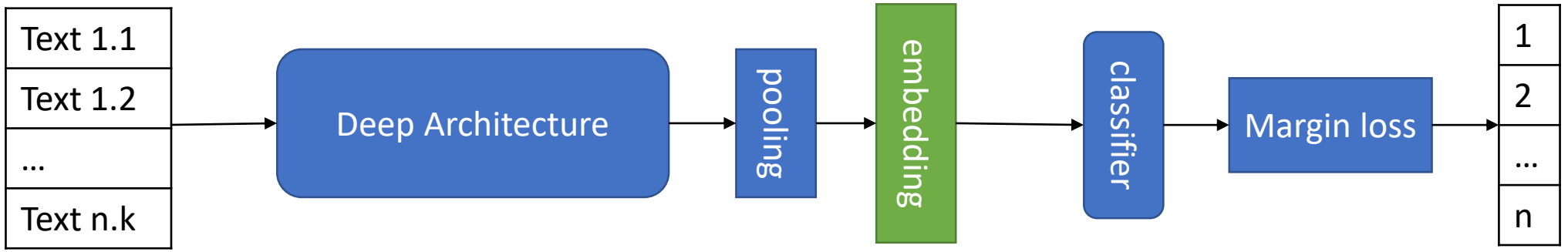
$$\|u - v\|_2^2 = 2 - 2s_{cos}(u, v)$$

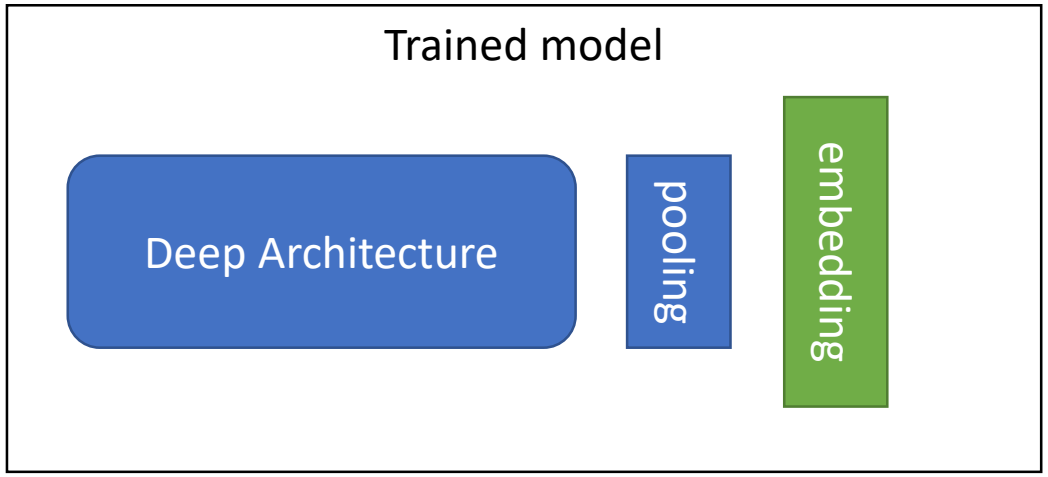
Plecháč, 2020, рис. 1 - arxiv.org/abs/2012.04796

Префиксная сепарация

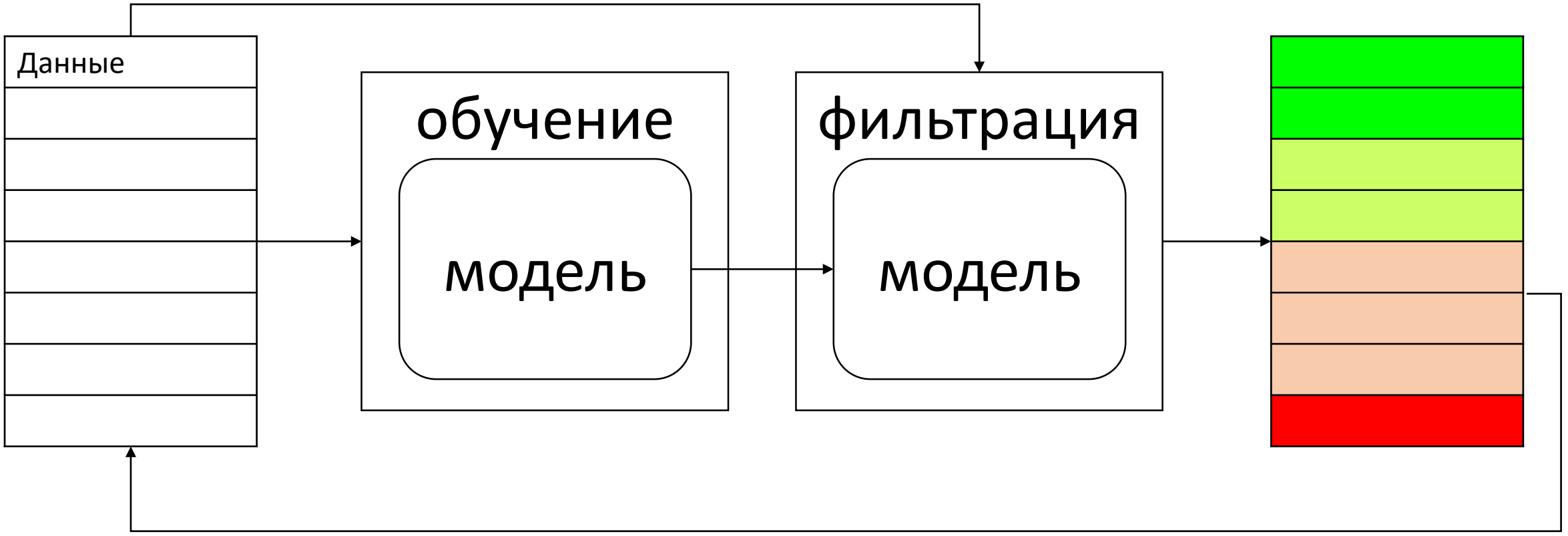
No prefix	
Сколько человек живут в Москве?	Численность населения Москвы 13 104 177 чел.
Task prefix	
[Question] Сколько человек живут в Москве?	[Answer] Численность населения Москвы 13 104 177 чел.
[Question] Сколько человек живут в Москве?	Численность населения Москвы 13 104 177 чел.
[Question][DSname] Сколько человек живут в Москве?	Численность населения Москвы 13 104 177 чел.
Instruction	
Ответь на вопрос: Сколько человек живут в Москве?	Численность населения Москвы 13 104 177 чел.
Soft prompt	
[Q1] [Q2] [Q3] Сколько человек живут в Москве?	Численность населения Москвы 13 104 177 чел.
[Q1] [Q2] [Q3] Сколько человек живут в Москве?	[s1] [s1] [s1] Численность населения Москвы 13 104 177 чел.

In-classifier pretraining





Active learning



Обучающие данные

Формат	**Пример**	**Источник сигнала**
Пара (q, d^+)	Запрос - релевантный документ	Остальные документы батча могут стать негативами
Тройка (q, d^+, d^-)	Запрос - позитив - негатив	Явно заданный контраст
Несколько позитивов	Запрос - несколько подходящих документов	Несколько корректных ответов

Метки задают смысл близости

Для IR позитив отвечает на информационную потребность запроса. Похожая тема сама по себе не делает документ релевантным.

Где q - запрос, d^+ - релевантный документ, d^- - негатив. Неразмеченный документ может оказаться релевантным.

Варианты разбиения данных

Сплит определяет, на что проверяем обобщение модели

Сплит	Что проверяем	Реальный пример
1. Общий корпус Разные запросы	Новые запросы к известной коллекции.	MS MARCO Passage: единая коллекция, запросы train / dev / test.
2. Общие запросы Разные корпусы	Новые документы для знакомых запросов.	TREC-COVID: общие темы раундов на новых версиях CORD-19.
3. Разные запросы и разные корпусы	Новые запросы и корпус. При смене тематики - перенос на новый домен.	BEIR: обучение на MS MARCO, тест на SciFact.

Версии корпуса могут пересекаться. Разметка релевантности относится к конкретному корпусу.

Источники: MS MARCO / TREC-COVID / BEIR

Краткие выводы

Представления и подготовка обучения

Вектор зависит от всего процесса кодирования

Pooling, префиксы, усечение и нормализация должны совпадать при обучении и поиске.

Функция подобия задаёт геометрию поиска

Cosine сравнивает направления. Скалярное произведение дополнительно учитывает длины векторов.

Позитивы задают смысл релевантности

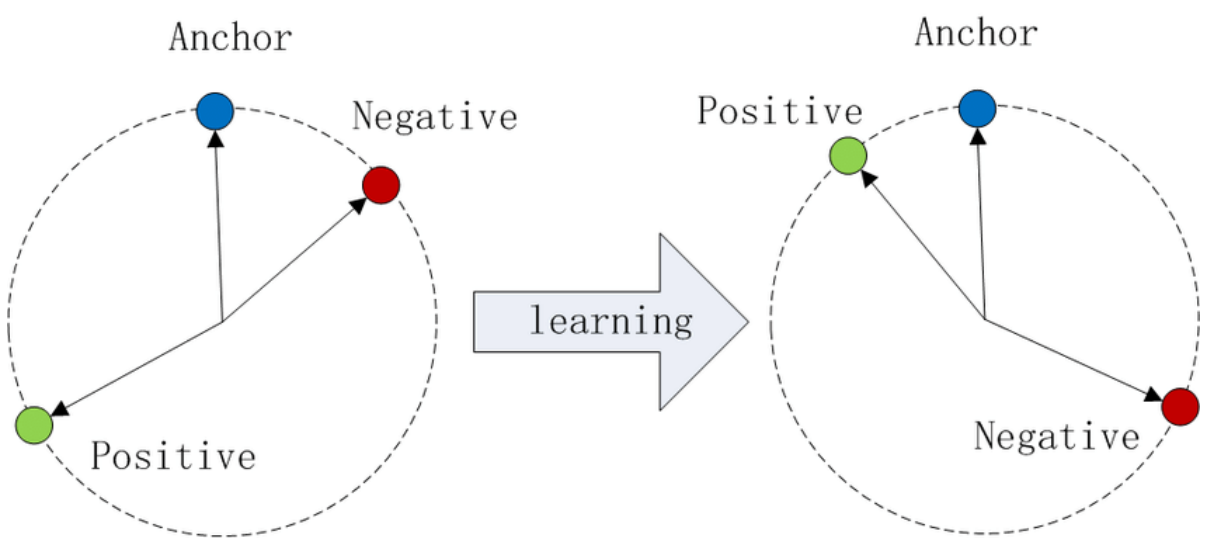
Обучающие пары должны соответствовать поисковой задаче. Совпадение темы само по себе не означает полезный ответ.

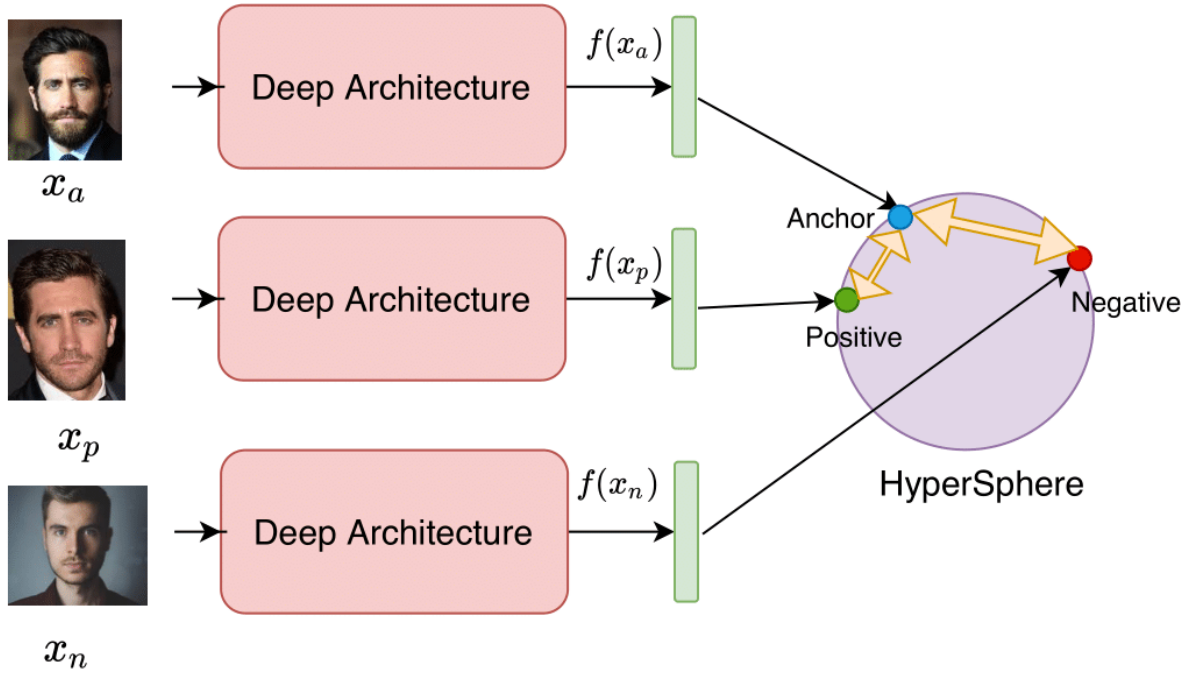
Сплит соответствует сценарию применения

Можно разделять запросы, корпуса или и то и другое. Выбранный вариант определяет, какое обобщение проверяем.

Функции потерь

Triplet loss





L(a, p, n) = max(0, D(a, p) — D(a, n) + margin)

a – anchor embedding

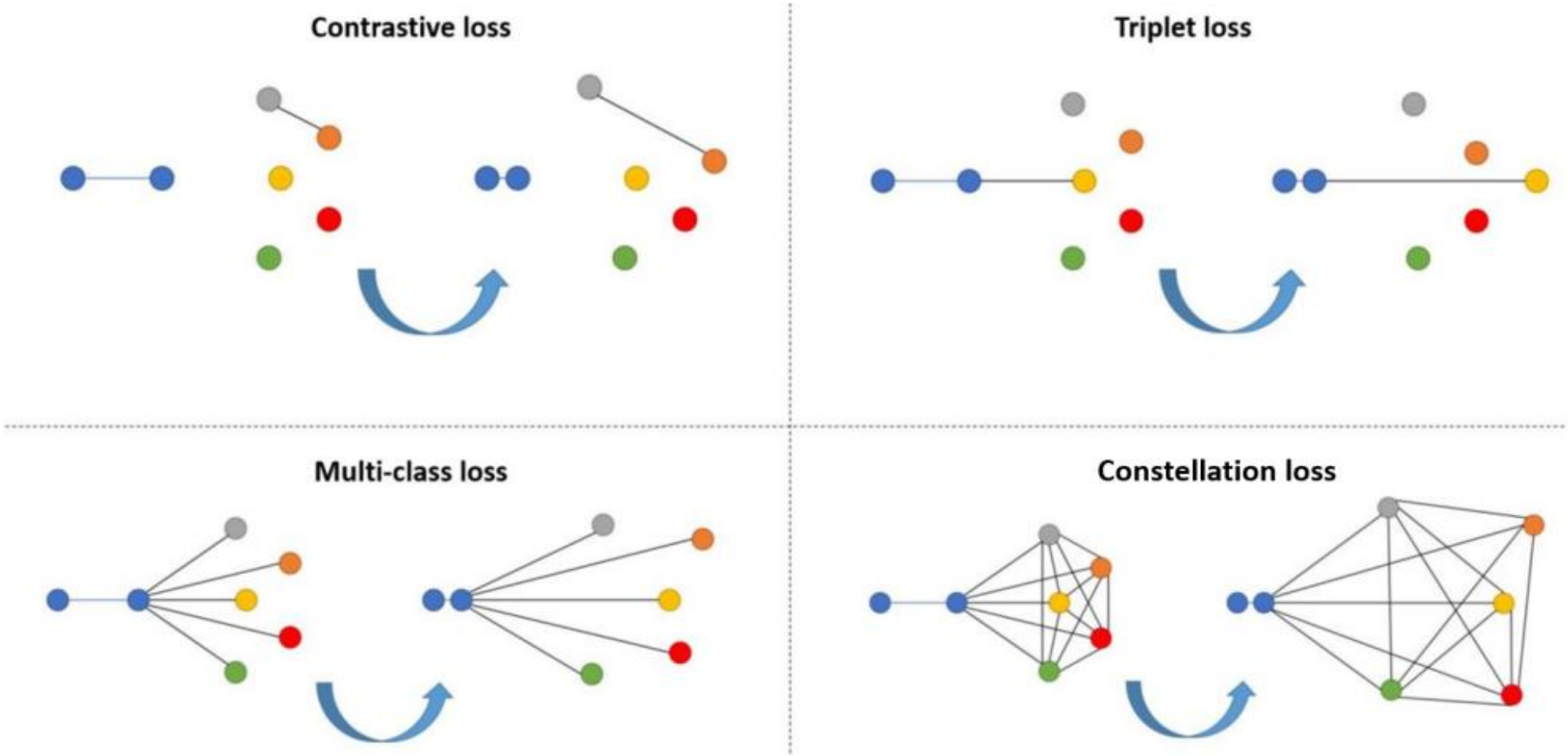
p – positive embedding

n – negative embedding

margin – margin between positive group and other samples

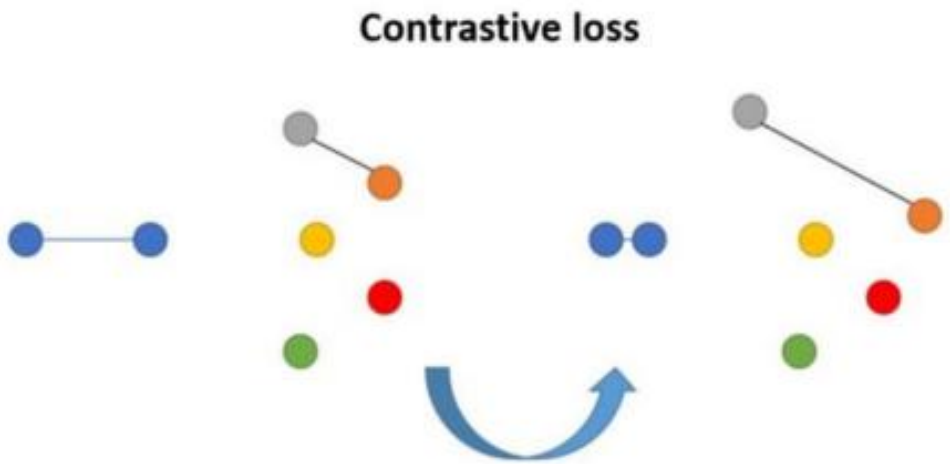
D – similarity function

Функции потерь: схемы



Схемы: Constellation Loss, рис. 1 - arxiv.org/abs/1905.10675

Contrastive loss: пары



Где

$f_{1,i}$, $f_{2,i}$ - эмбеддинги i-й пары.

$y_i = 0$ - позитивная пара;
$y_i = 1$ - негативная пара.

m - минимальное расстояние для негативной пары.

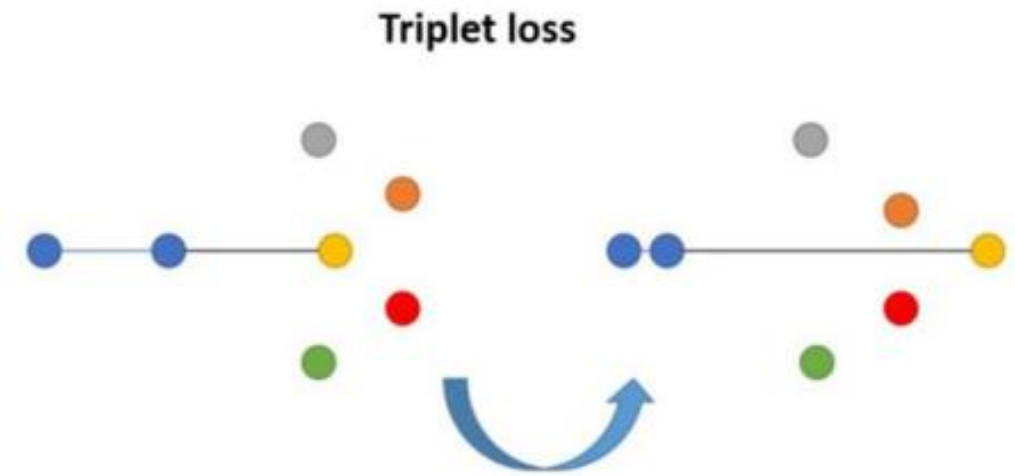
N - число пар; i - индекс пары.

$\|\cdot\|_2$ - евклидова норма.

$$\mathcal{L}_c = \frac{1}{2N}\sum_{i=1}^{N}[(1-y_i)||f_{1,i}-f_{2,i}||_2^2 + (y_i)\{max(0, m-||f_{1,i}-f_{2,i}||_2)\}^2] \qquad (1)$$

Medela, Picon. Constellation Loss, 2019, формула (1) - arxiv.org/abs/1905.10675

Triplet loss



Где

f_i^a, f_i^p, f_i^n - эмбеддинги anchor, позитива и негатива i-го триплета.

Позитив - того же класса, негатив - другого класса.

α - зазор между квадратами расстояний до позитива и негатива.

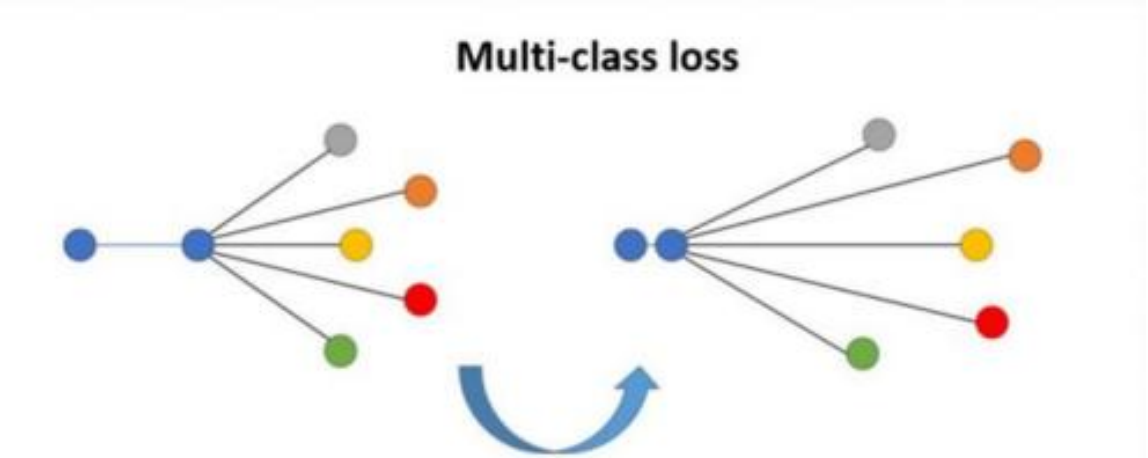
N - число триплетов; i - их индекс.

$\|\cdot\|_2^2$ - квадрат евклидовой нормы.

$$\mathcal{L}_{triplet} = \frac{1}{N}\sum_{i=1}^{N} max(0, ||f_i^a - f_i^p||_2^2 - ||f_i^a - f_i^n||_2^2 + \alpha) \qquad (2)$$

Medela, Picon. Constellation Loss, 2019, формула (2) - arxiv.org/abs/1905.10675

Multiclass loss



Где

f_i - эмбеддинг anchor i-й пары;
f_i^+ - эмбеддинг его позитива.

f_j^+ - позитив другой пары;
для anchor i он служит негативом.

N - число пар разных классов
в батче; всего в нём 2N объектов.

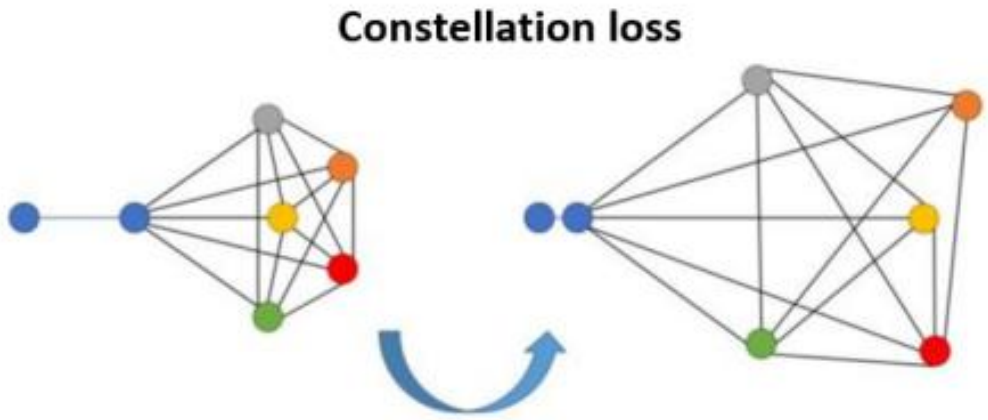
i, j - индексы пар; $j \neq i$.

$^\top$ - транспонирование;
$f_i^\top f_j^+$ - скалярное произведение.

$$\mathcal{L}_{m-c} = \frac{1}{N} \sum_{i=1}^{N} log(1 + \sum_{j \neq i} exp(f_i^\top f_j^+ - f_i^\top f_i^+)) \quad (3)$$

Medela, Picon. Constellation Loss, 2019, формула (3) - arxiv.org/abs/1905.10675

Constellation loss



Где

f_i^a, f_i^p - эмбеддинги anchor
и его позитива (того же класса).

f_j^n - эмбеддинг j-го негатива
другого класса для текущего anchor.

K - число негативов на anchor;
j - индекс негатива, от 1 до K.

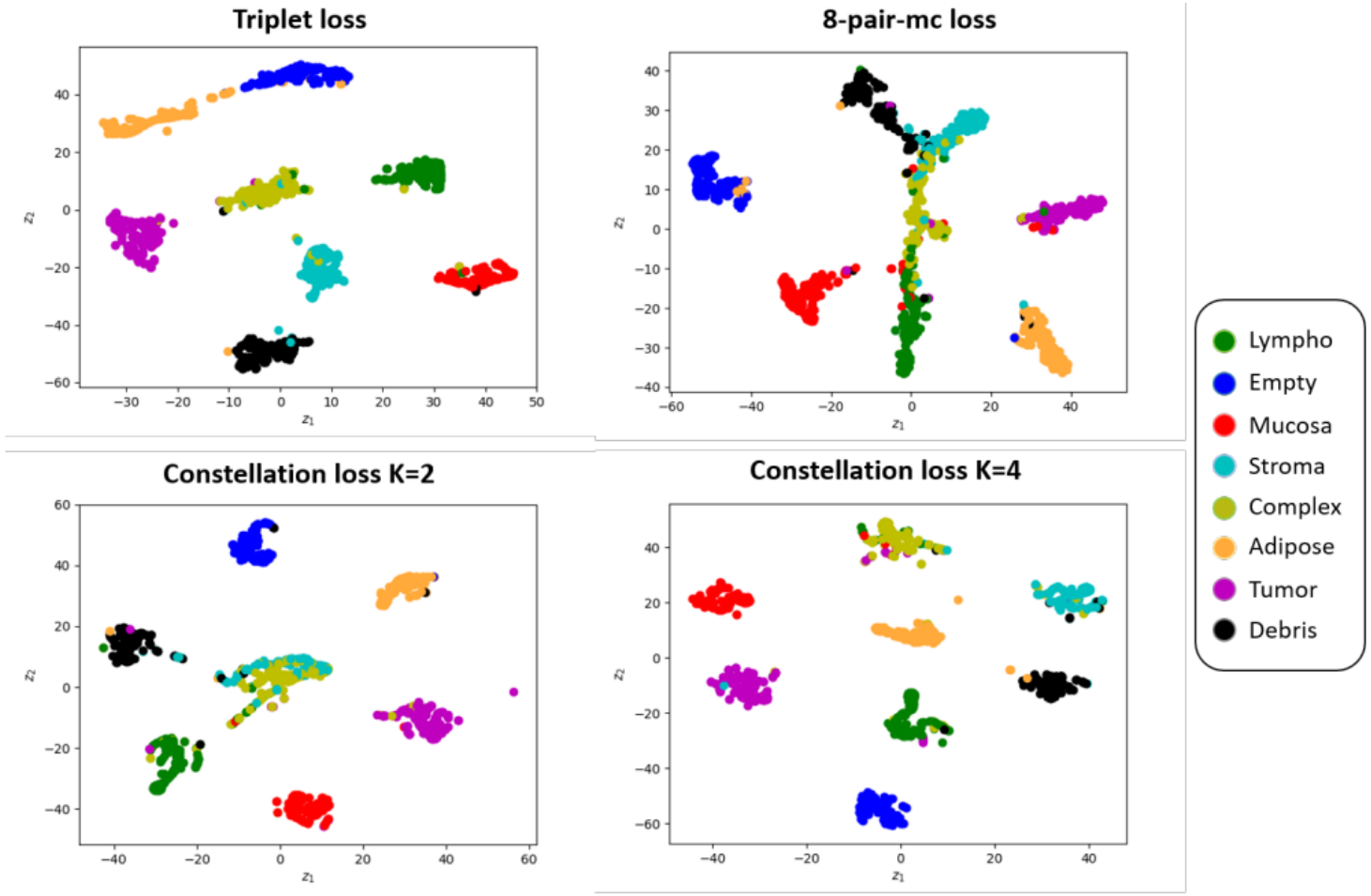
N - число anchor-позитив пар;
i - индекс такой пары.

$^\top$ - транспонирование;
$f_i^{a\top} f_j^n$ - скалярное произведение.

$$\mathcal{L}_{constellation} = \frac{1}{N} \sum_{i=1}^{N} log(1 + \sum_{j}^{K} exp(f_i^{a\top} f_j^n - f_i^{a\top} f_i^p)) \quad (4)$$

Medela, Picon. Constellation Loss, 2019, формула (4) - arxiv.org/abs/1905.10675

Геометрия эмбеддингов после обучения



Constellation Loss, рис. 7 - arxiv.org/abs/1905.10675

InfoNCE

$$InfoNCE = -\log\left(\frac{e^{\phi(q,c_i^+)\tau}}{e^{\phi(q,c_i^+)\tau} + \sum_{j=1,j\neq i}^{N} e^{\phi\left(q,c_j^-\right)\tau}}\right),$$

где q – векторное представление запроса;

c_i^+ – векторное представление позитивного кандидата i;

c_j^- – векторное представление негативного кандидата j;

N – размер обучающего пакета;

τ – константа, повышающая контрастность, также называемая температурой;

$\phi(q, c_i^+)$ – функция подобия

$$CosSim(q,c) = \frac{q \cdot c}{\|q\| \cdot \|c\|},$$

где c – векторное представление произвольного кандидата.

Краткие выводы

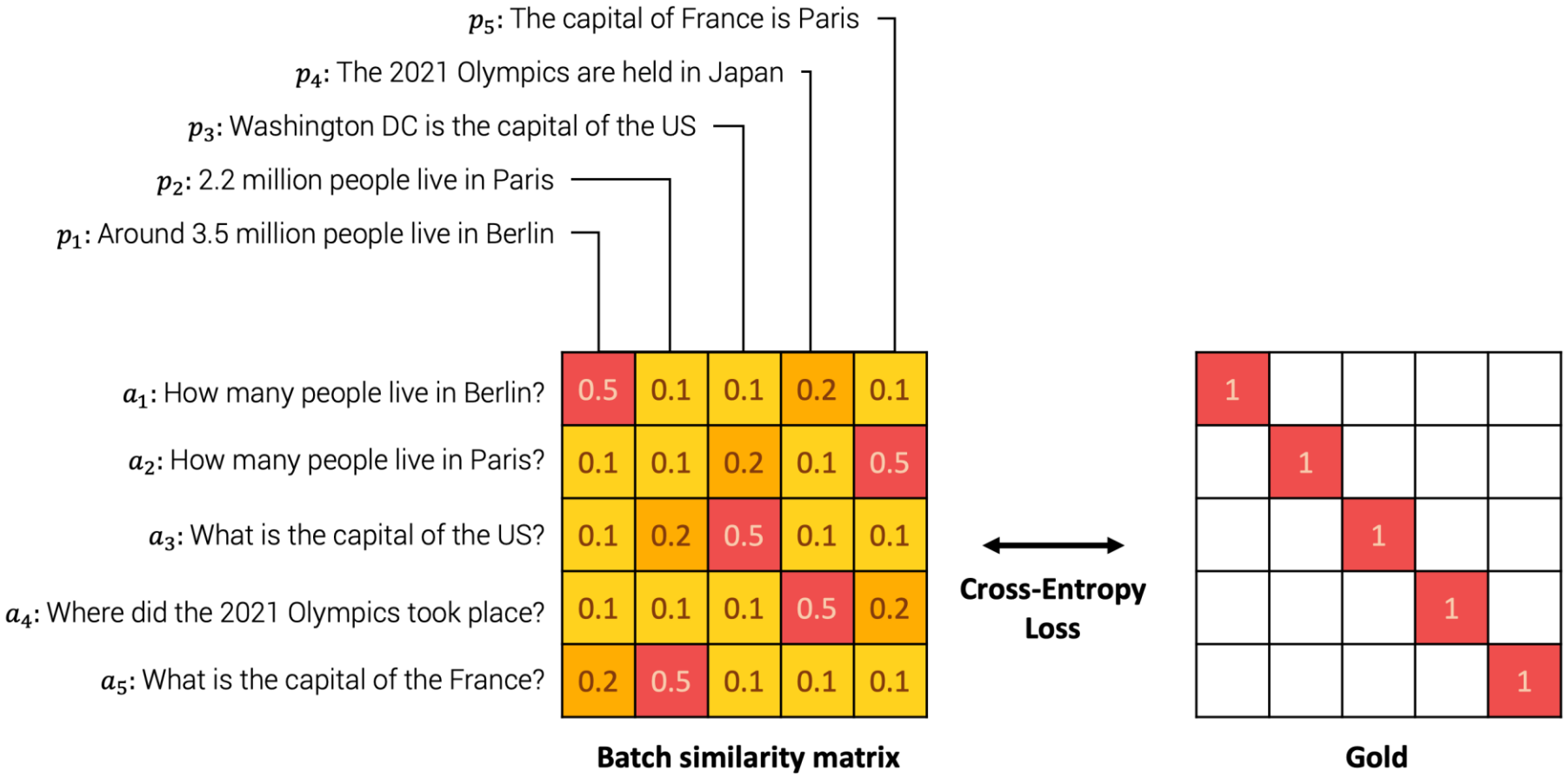
Функции потерь

Loss	**Обучающий сигнал**
Contrastive	Сближает позитивную пару. Отталкивает негативную, пока расстояние меньше зазора.
Triplet	Задаёт относительный зазор между позитивом и одним негативом.
N-pair / Constellation	Сравнивают позитив с несколькими негативами. Различаются выбором негативов и устройством батча.
InfoNCE	Повышает долю позитива в softmax по кандидатам. Температура задаёт чувствительность к разницам скоров.

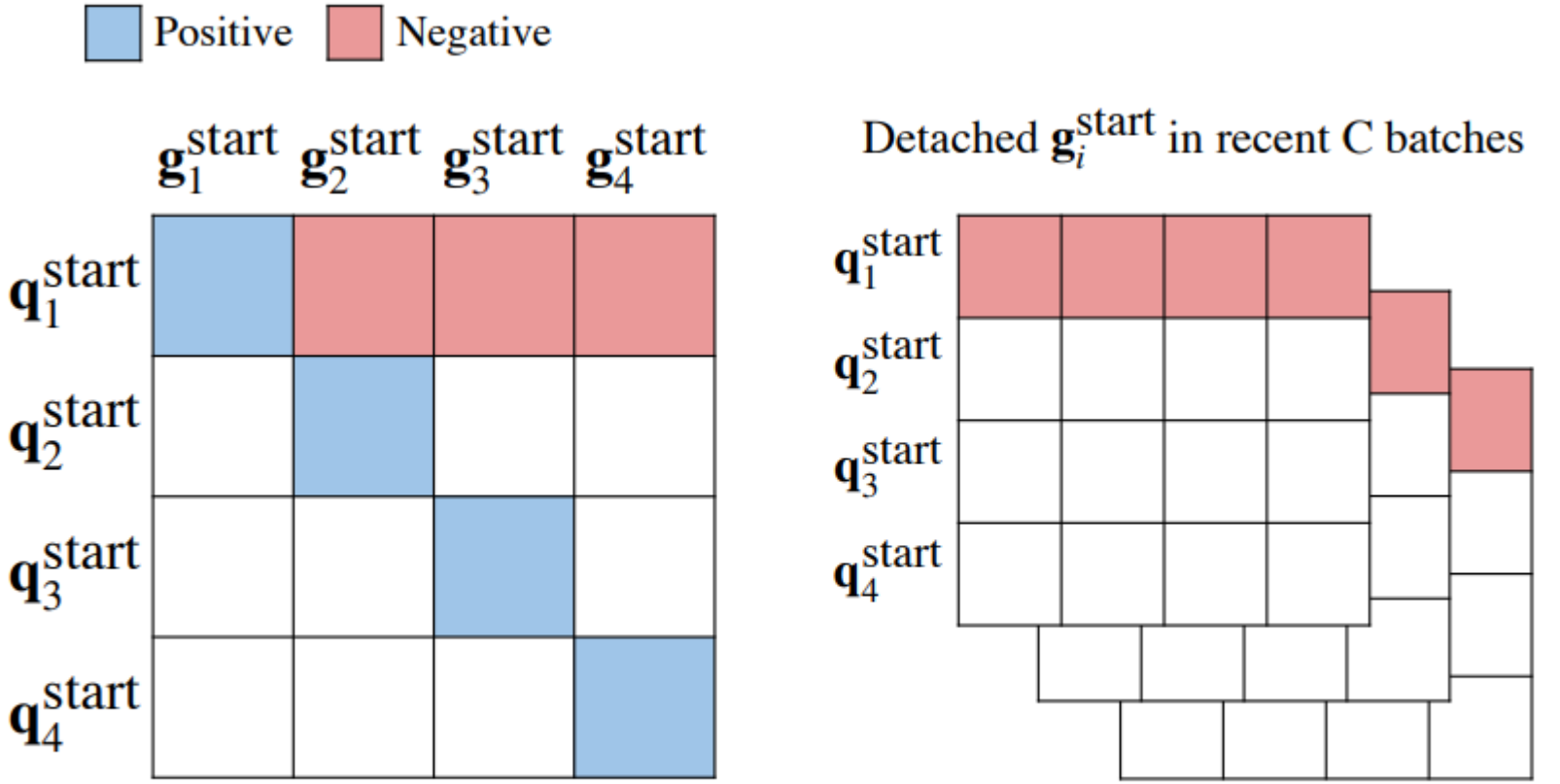
Форма loss и выбор негативов вместе определяют сигнал обучения.

Негативное семплирование

In-Batch negative sampling

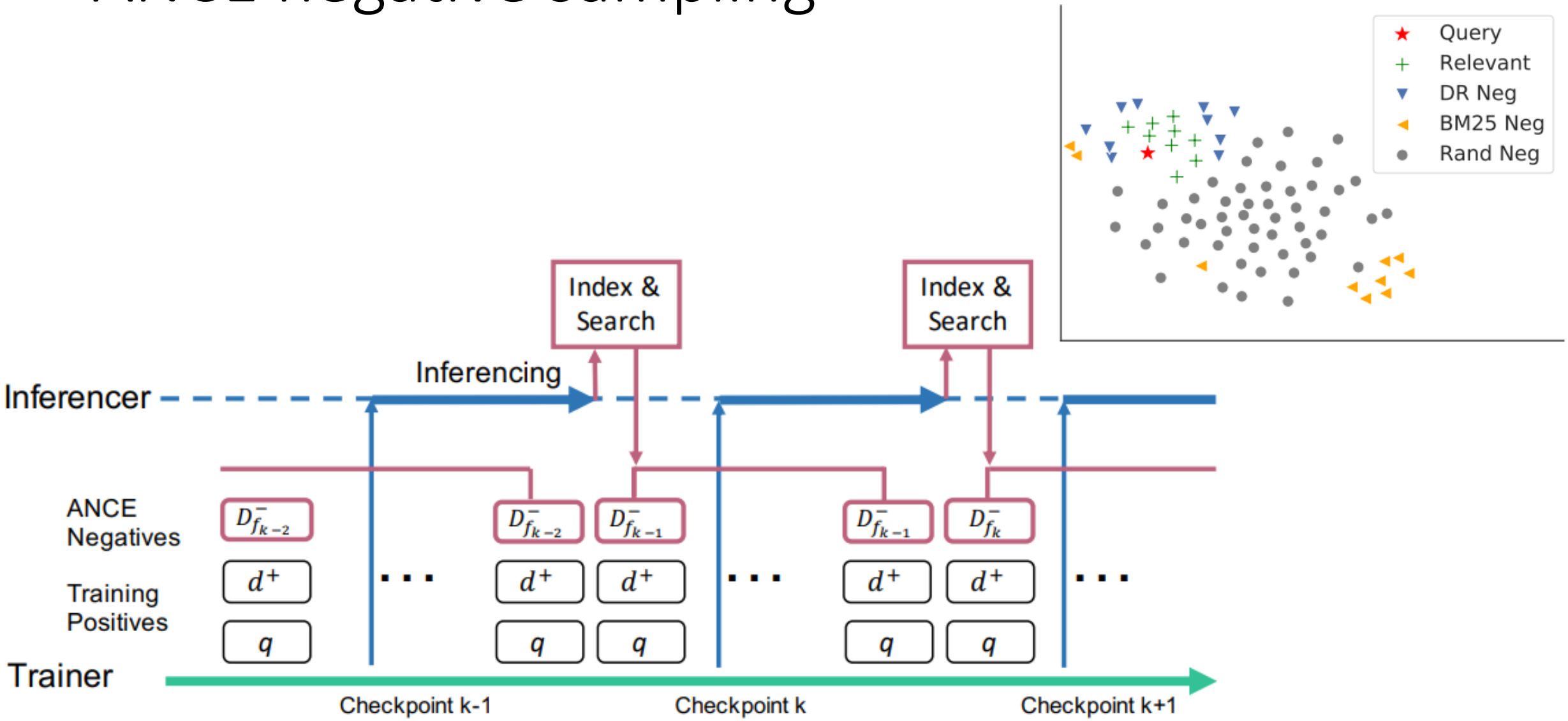


Pre-Batch negative sampling



(a) In-batch Negatives ($B-1$) (b) Pre-batch Negatives ($B \times C$)

ANCE negative sampling



Сложные и ложные негативы

Тип негатива	Как получить	Что учитывать
Случайный	Случайный документ корпуса	Часто слишком легко отличить от позитива
In-batch	Документ другой пары	Может быть релевантен текущему запросу
Hard negative	Высокий скор у нерелевантного документа	Даёт полезный сигнал, но требует проверки меток

Ложный негатив противоречит задаче

Если релевантный документ пометить как негатив, модель будет учиться снижать его скор. Из негативов исключают известные позитивы и дубликаты.

CLIP loss

```
# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Figure 3. Numpy-like pseudocode for the core of an implementation of CLIP.

CLIP loss

	Candidate 1	Candidate 2	Candidate 3
Query 1	1	0	0
Query 2	0	1	0
Query 3	0	0	1

	Query 1	Query 2	Query 3
Candidate 1	1	0	0
Candidate 2	0	1	0
Candidate 3	0	0	1

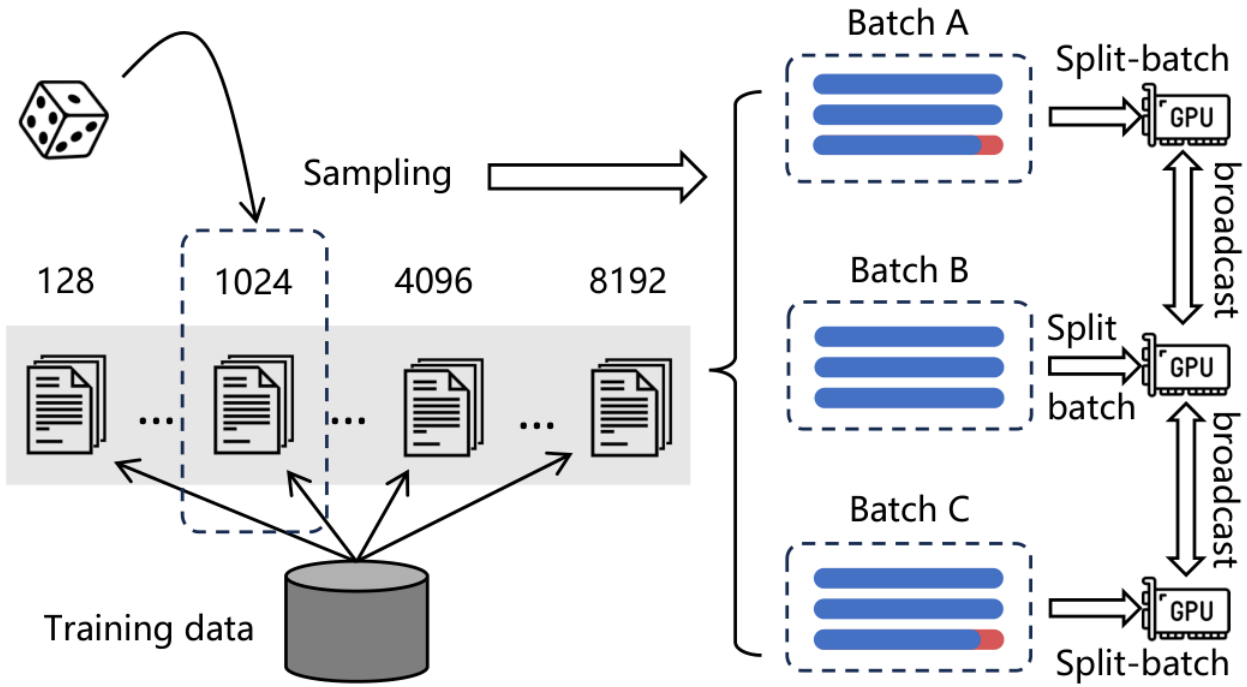
	Candidate 1	Candidate 2	Candidate 3
Query 1	0.5	-0.8	0.3
Query 2	0.6	0.4	-0.9
Query 3	0.5	0.1	0.9

	Query 1	Query 2	Query 3
Candidate 1	0.5	0.6	0.5
Candidate 2	-0.8	0.4	0.1
Candidate 3	0.3	-0.9	0.9

$$L = \frac{1}{2}[\mathrm{CE}(S, y) + \mathrm{CE}(S^T, y)]$$

CE - cross-entropy по строкам скоров S. Метки y задают правильные пары.

Batch split: BGE-M3



Кодирование идёт частями.

Loss использует эмбеддинги всего батча.

[BGE-M3, оригинальный рис. 3 и приложение B.3 - arxiv.org/abs/2402.03216]

Группировка по длине

Похожие длины текстов уменьшают padding.

Split-batch с checkpointing

Энкодер обрабатывает части последовательно.

Общий пул кандидатов

Эмбеддинги частей объединяются и передаются между GPU.

Batch split: память

	Без checkpointing	**С checkpointing**
Forward	Autograd сохраняет активации каждой части до общего backward.	Сохраняет входы и checkpoints. Часть активаций не хранится.
Backward	Использует сохранённые активации всех частей.	Пересчитывает нужные активации. Расход вычислений возрастает.

$$M_{act} \approx \sum_{j=1}^{m} A_j$$

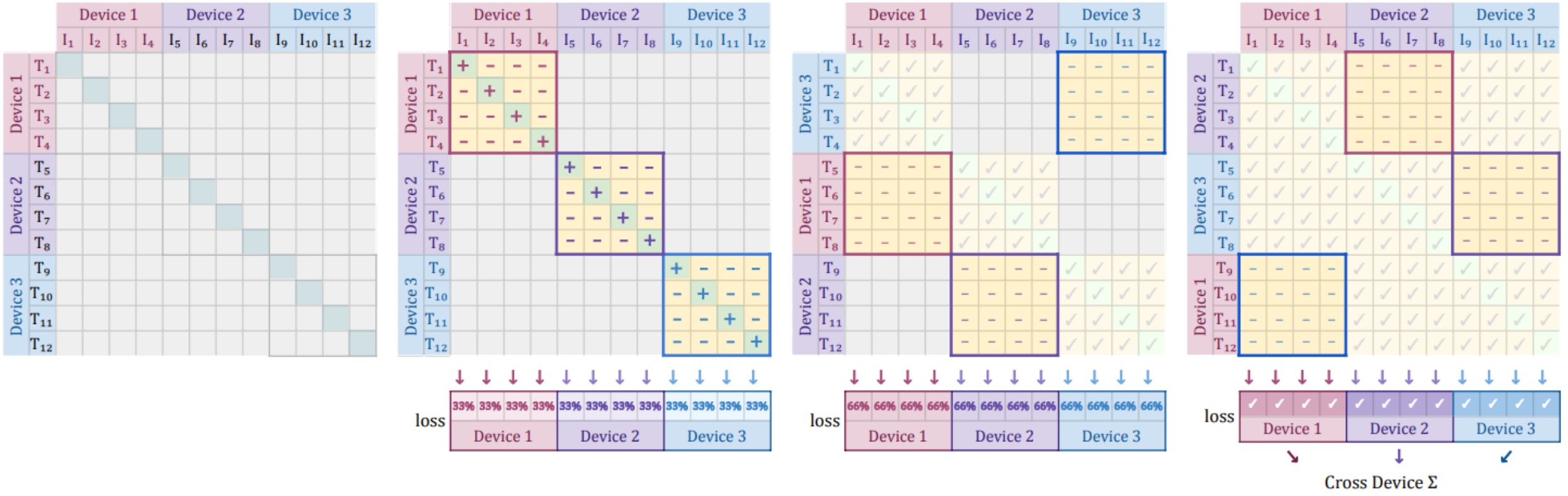
$$M_{act} \approx \sum_{j=1}^{m} C_j + max_j R_j$$

Где A_j - активации части, C_j - её checkpoints, R_j - временные активации пересчёта.
Здесь показана только память активаций. Эмбеддинги и матрица скоров остаются.

Без checkpointing основная память активаций накапливается, поэтому выигрыш split мал.

BGE-M3, приложение B.3 - Chen et al., 2016: arxiv.org/abs/1604.06174

Chunked SigLIP



Цикл обучения Bi-Encoder

1. Собираем батч пар «запрос - позитив» и удаляем конфликтующие дубликаты.
2. Кодируем запросы и документы, применяем pooling и нормализацию.
3. Строим матрицу скоров и вычисляем контрастивный loss.
4. Выполняем backward и шаг оптимизатора, обновляя общие веса.
5. На validation ищем по корпусу и сохраняем checkpoint с лучшей метрикой.

Валидация использует текущую модель

После обновления энкодера векторы корпуса нужно пересчитать. Старые векторы уже не соответствуют новым параметрам.

Параметры обучения

Параметр	Влияние на обучение
Learning rate	Слишком большой шаг может разрушить полезные представления
Размер batch	Определяет число in-batch негативов и расход памяти
Максимальная длина	Ограничивает видимый текст и вычислительную стоимость
Температура / scale	Меняет относительный вклад близких кандидатов в loss
Число шагов и эпох	Задаёт объём обучения и риск переобучения

Параметры и checkpoint выбирают по validation-метрике. Для сравнения сохраняют seed, split, корпус и конфигурацию.

Метрики

Ранжирование, классификация, калибровка

Протокол оценки

Корпус, кандидаты и финальная выдача

$$O_q \subseteq A_q \subseteq C$$

Где C - корпус, A_q - найденные кандидаты, O_q - результат после усечения.
Qrels связывают query_id, doc_id и метку релевантности.

Что фиксируем	**Правило**
Сравнение моделей	Один test-набор запросов, корпус, разметка и глубина выдачи.
Порог и калибровка	Настроить на validation. На test применять без подбора.
Агрегация	Macro: среднее по запросам. Micro: сначала суммировать TP, FP, FN.

Неизвестная релевантность не равна метке 0. Неполные qrels ограничивают оценку.

Оценка IR - Manning, Raghavan, Schütze, глава 8

Метрики ранжирования

Метрика	Что измеряет	Разметка
Precision@k	Долю релевантных в первых k результатах	Бинарная
Recall@k	Долю найденных релевантных	Бинарная
MRR@k	Позицию первого релевантного	Бинарная
MAP@k	Precision на позициях релевантных, усреднённую по запросам	Бинарная
nDCG@k	Полезность выдачи с учётом ранга и степени релевантности	Бинарная или градуированная

Cutoff k задают по сценарию: глубина выдачи пользователю или число кандидатов реранкера.

Метрики ранжирования - Manning, Raghavan, Schütze, глава 8

Precision@k и Recall@k

Precision@k: точность выдачи

$$P@k = \frac{\sum_{i=1}^{k} y_i}{k}$$

Доля релевантных среди k позиций.
Высокое значение означает мало шума.

Recall@k: полнота поиска

$$R@k = \frac{\sum_{i=1}^{k} y_i}{|R_q|}$$

Доля найденных релевантных.
Знаменатель включает пропущенные.

Где $y_i \in \{0, 1\}$ - метка на ранге i, R_q - релевантные документы запроса в корпусе.

Ранг	**1**	**2**	**3**	**4**	**5**
Метка y_i	0	1	1	0	1

В корпусе 5 релевантных. При k = 3: Precision = 2/3, Recall = 2/5.

Precision и Recall - Manning, Raghavan, Schütze, глава 8

MRR@k

Средний обратный ранг первого релевантного

$$\mathrm{MRR@}k = \frac{1}{|Q|} \sum_{q \in Q} \mathrm{RR@}k(q)$$

Где Q - множество запросов; RR@k(q) = 1/r, если первый релевантный стоит на ранге r ≤ k. Если его нет в top-k, RR@k(q) = 0.

Запрос	**Первый релевантный**	**RR@5**
q_1	Ранг 1	1
q_2	Ранг 3	1/3
q_3	Нет в top-5	0

MRR@5 = (1 + 1/3 + 0) / 3 ≈ 0,444. Остальные релевантные на метрику не влияют.

MRR - Sentence Transformers: InformationRetrievalEvaluator

nDCG@k

DCG: полезность с учётом позиции

$$DCG@k = \sum_{i=1}^{k} \frac{2^{g_i} - 1}{\log_2(i+1)}$$

$$nDCG@k = \frac{DCG@k}{IDCG@k}$$

Где $g_i \geq 0$ - степень релевантности на ранге i, IDCG@k - DCG идеального порядка.
Ранние и более релевантные документы дают больший вклад.

Порядок	**Ранг 1**	**Ранг 2**	**Ранг 3**	**DCG@3**
Фактический	0	3	1	4,917
Идеальный	3	2	1	9,393

nDCG@3 ≈ 0,523. При IDCG > 0 значение лежит от 0 до 1, идеальный порядок даёт 1.

DCG и nDCG - Manning, Raghavan, Schütze, глава 8

Классификация по порогу

Решение для размеченной пары

$$\hat{y}_i = 1[s_i \geq t]$$

Где s_i - скор пары, t - порог.
$\hat{y}_i = 1$ означает «оставить документ».

Решение	**y = 1**	**y = 0**
$\hat{y} = 1$	TP	FP
$\hat{y} = 0$	FN	TN

Метрики на этой выборке

$$P = \frac{TP}{TP + FP}$$

$$R = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2TP}{2TP + FP + FN}$$

$$Acc = \frac{TP + TN}{TP + FP + FN + TN}$$

Accuracy может быть высокой из-за большого числа нерелевантных документов.

Метрики классификации - scikit-learn: Model evaluation

Классификация усечённой выдачи

Пропуски за пределами кандидатов

$$FN_C = FN_A + FN_{miss}$$

Где FN_A - релевантные кандидаты, отброшенные порогом.
FN_{miss} - релевантные документы, не попавшие в кандидаты.

Пример: 10 документов в корпусе

Группа	**Рел.**	**Нерел.**
Оставлены порогом	2 (TP)	1 (FP)
Кандидаты, отброшены	1 (FN)	1 (TN)
Вне кандидатов	2	3

$$R_A = \frac{2}{2+1} = \frac{2}{3}$$

$$R_C = \frac{2}{2+1+2} = \frac{2}{5}$$

Precision = 2/3 в обоих случаях. F1: 2/3 на кандидатах, 1/2 по всему корпусу.
Разметка только оставленного top-k позволяет оценить Precision, но скрывает пропуски.

Уровни оценки: кандидаты и весь корпус - определения IR

ROC-AUC и PR-AUC

ROC: TPR в зависимости от FPR

$$TPR = \frac{TP}{TP + FN}, FPR = \frac{FP}{FP + TN}$$

Меняем порог и измеряем площадь под ROC-кривой.
AUC = 1 соответствует идеальному разделению классов.

PR: Precision при разном Recall

$$\mathrm{AP} = \sum_j (R_j - R_{j-1}) P_j$$

AP суммирует прирост Recall с весом Precision.
AP и площадь по трапециям под PR-кривой могут различаться.

Где P_j и R_j - Precision и Recall при последовательном снижении порога.
Для ROC-AUC в оценочной выборке должны присутствовать оба класса.

На top-k эти метрики описывают только выбранных кандидатов.
Сравнение требует одного пула оценки. PR-метрики зависят от доли релевантных.

scikit-learn - roc_auc_score и average_precision_score

Корреляция Пирсона

Линейная согласованность скоров и числовых меток

$$r = \frac{\sum_{i=1}^{n}(s_i - m_s)(y_i - m_y)}{\sqrt{\sum_{i=1}^{n}(s_i - m_s)^2}\sqrt{\sum_{i=1}^{n}(y_i - m_y)^2}}$$

Где s_i - скор пары, y_i - числовая метка сходства или релевантности.
m_s и m_y - средние значения, n - число размеченных пар.

Метка y	0	1	2	3
Скор s	10	12	14	16

Здесь r = 1, хотя значения скоров не совпадают со шкалой меток.

r ∈ [-1, 1]. При постоянных s или y коэффициент не определён.

SciPy - scipy.stats.pearsonr

Корреляция Спирмена

Согласованность порядка при нелинейной связи

$$\rho = \mathrm{corr}(\mathrm{rank}(s), \mathrm{rank}(y))$$

Где rank - ранги значений по возрастанию, corr - корреляция Пирсона.
Одинаковым значениям присваивается средний ранг.

Метка y	**0**	**1**	**2**	**3**
Скор s	0	1	4	9
Ранги обоих	1	2	3	4

Здесь ρ = 1, а r ≈ 0,958. Спирмен учитывает порядок, Пирсон - линейность.

Для оценки внутри запроса считайте r и ρ по запросам. Постоянные ряды исключают с отчётом.

SciPy - scipy.stats.spearmanr и rankdata

Калибровка вероятностей

Вероятность и частота релевантности

$$P(y = 1 \mid p) \approx p$$

Где p - предсказанная вероятность.
При p ≈ 0,8 ожидаем около 80% релевантных.

Ошибка по группам вероятностей

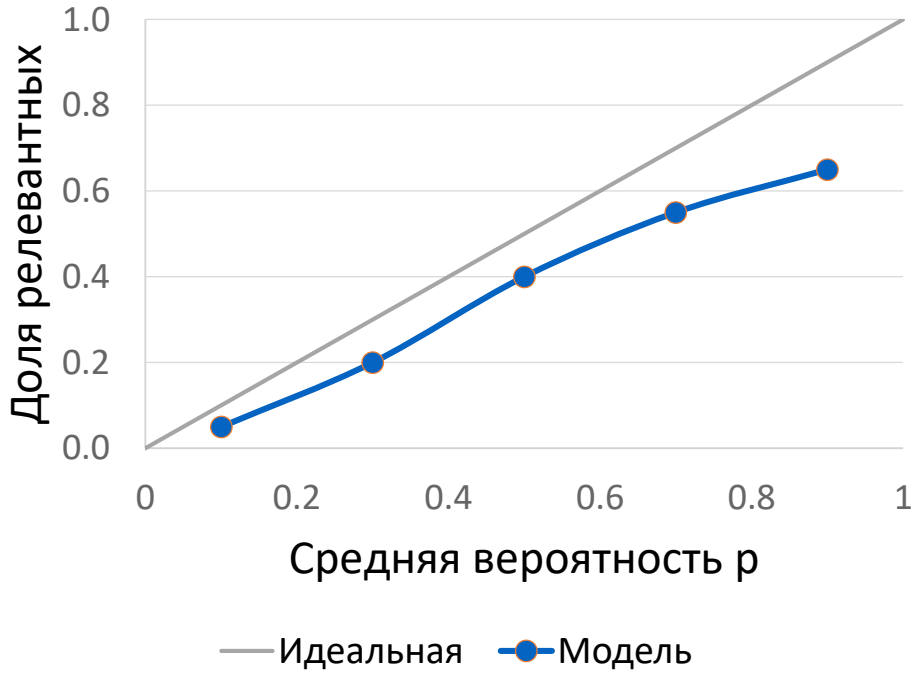
$$ECE = \sum_{b=1}^{B} \frac{n_b}{n} |f_b - p_b|$$

Где B - группы, n - всего пар, n_b - в группе.
p_b - средняя вероятность, f_b - доля релевантных. ECE зависит от группировки.

Высокая корреляция не гарантирует калибровку.

Калибровка - scikit-learn: Probability calibration

Пример: вероятности завышены



Пять равных групп: ECE = 0,13.

Краткие выводы

Оценка поисковой модели

Порядок и полнота оцениваются разными метриками

MRR и nDCG учитывают позиции. Recall по корпусу включает релевантные документы, которые не попали в кандидаты.

Порог превращает ранжирование в классификацию

Precision и F1 зависят от порога и состава выборки.
Порог выбирают на validation, затем фиксируют для test.

Корреляции характеризуют согласованность скоров

Пирсон оценивает линейную связь с метками, Спирмен - порядок.
Высокая корреляция не гарантирует калибровку вероятностей.

Сравнение требует одного протокола

Корпус, разметка, cutoff и способ усреднения должны совпадать.
Средние значения дополняют срезами и разбором отдельных запросов.

Анализ ошибок

Наблюдение	**Возможная причина**	**Что проверить**
Recall на кандидатах высокий, по корпусу низкий	Релевантные документы не попали в кандидаты	Глубину retrieval, индекс и полноту qrels
MRR высокий, Recall низкий	Первый ответ найден, остальные потеряны	Запросы с несколькими релевантными документами
Ранжирование хорошее, Precision по порогу падает	Скоры разных запросов имеют разный масштаб	Порог и калибровку на целевом потоке
Метрика растёт только на случайных негативах	Оценочная выборка слишком простая	Фиксированный корпус и трудные кандидаты
Средняя метрика стабильна, часть запросов деградирует	Усреднение скрывает неоднородность	Срезы по языку, длине, теме и типу запроса

Для каждого разбора сохраняйте запрос, выдачу, скоры, метки и пропущенные релевантные.

Итоги лекции

Архитектура определяет способ сравнения и его стоимость

Bi-Encoder позволяет кодировать документы заранее. Взаимодействующие модели добавляют сопоставление представлений запроса и документа.

Данные определяют, что модель считает релевантным

Разметка, префиксы и подготовка текста должны соответствовать задаче поиска.

Loss и негативы совместно задают обучающий сигнал

Информативные сравнения помогают различать близкие по теме документы.
Ложные негативы противоречат целевой релевантности.

Организация батча связывает качество и расход памяти

Batch split с checkpointing сохраняет общий пул сравнений при кодировании частями.

Оценка должна соответствовать конечной поисковой системе

Проверяют порядок, полноту, решения по порогу и свойства скоров.

Лабораторная 1: обучение Bi-Encoder

Цель: дообучить модель поиска и получить **+1 п.п. nDCG@10** к исходному checkpoint.

Модель и данные

intfloat/e5-small
FiQA2018 или NFCorpus из MTEB.

Можно выбрать свою модель или свой датасет.

Два варианта реализации

Свой пайплайн на **PyTorch Lightning** или готовый код **FlagEmbedding**:

finetune/embedder/encoder_only/base

E5: mean pooling, L2-нормализация, префиксы «query: » и «passage: ».

train - обучение; **dev** - выбор настроек; **test** - итоговая оценка.
Используем готовые сплиты запросов и общий полный корпус.

Ссылки: E5-small · FiQA · NFCorpus · код encoder_only/base · пример запуска

Лабораторная 1: эксперименты

8 комбинаций (2 × 2 × 2) + baseline без дообучения

Фактор	**Вариант 1**	**Вариант 2**
Негативы	Случайные, без hard	Hard negatives
Контрастивный батч B	Малый: 16 запросов	Большой: 64 запроса
Температура τ	Низкая: 0,01	Высокая: 0,05

B и τ - рекомендуемые значения. В обоих режимах оставляем in-batch негативы и одинаковое число явных негативов на запрос.

Фиксируем остальные настройки и объём обучения. B - число запросов в одном loss; обычный gradient accumulation не увеличивает этот пул.

FlagEmbedding: формат данных · hard negatives · batch и temperature в base.sh

Результат лабораторной

nDCG@10 после - nDCG@10 до ≥ 0,01 на test
Например: 0,350 → 0,360, то есть +1 п.п. по шкале 0-100.

Оценка и эксперименты

Точный поиск по полному корпусу.
Единый протокол до и после обучения.

Таблица 8 запусков на dev.
На test: baseline и лучший по dev.

nDCG@10, MRR@10, Recall@10.
Выводы по трём факторам
и примеры изменившейся выдачи.

Что сдаём

Код, конфиги, seed, версии библиотек
и команды воспроизведения.

Checkpoint и токенизатор;
pooling, префиксы, длины входов.

Корпус, qrels, сплиты и векторы
с doc_id - для следующей
лабораторной с Qdrant.

Параметры и ссылки для реализации приведены в приложении к подсказкам лектору.
FlagEmbedding: настройки обучения · encoder_only/base · скрипт mining негативов